

Design-Space Exploration and Integer Quantization of Graph Neural Networks for Real-Time FPGA Track Finding

Pelayo Leguina

IEEE Real-Time 2026 (25–29 May 2026), La Biodola – Isola d’Elba, Italy

Motivation and Context

Fixed-latency trigger/DAQ systems in the CMS Level-1 upgrade (12.5 μ s budget [2]) demand inference architectures that balance accuracy with hardware feasibility under strict throughput and resource constraints. Displaced-muon signatures from long-lived particles pose unique challenges for traditional pattern-based track finding because they deviate from prompt-track assumptions. Graph neural networks (GNNs) naturally represent sparse, irregular detector geometries: nodes encode stub-level features and edges capture local geometric compatibility, making them well-suited for the overlap muon track finder region [3] where barrel and endcap detectors meet.

Methodology and Workflow

This work develops a reproducible pipeline from PyTorch Geometric model training to FPGA-ready HLS implementations. Unlike automated ML toolflows that primarily target dense or convolutional layers, message-passing GNN operations require explicit handling of irregular aggregation patterns, dynamic data movement, and careful numeric design. Our framework includes: (i) multiple HLS implementation paths (floating-point baseline, PTQ-Float hybrid, and pure integer PTQ-INT8); (ii) automated design-space exploration (DSE) across architectural, quantization, and microarchitectural parameters; (iii) data-driven bit-width optimization for minimizing resource usage while maintaining numerical correctness.

Model Training and Quantization

GraphSAGE architecture: feature projection (1433 $\rightarrow$ 16), two message-passing layers (16 $\rightarrow$ 24 $\rightarrow$ 7). *Base model* (Cora, 1000 nodes): 80.3% accuracy. *Reduced variant* (24K parameters, FPGA-compatible): 75.8% accuracy. *Post-training quantization (PTQ)* to INT8 maintains 75.7% accuracy (0.1% degradation). *Quantization-aware training (QAT)* is under active development; preliminary experiments show 28.2% accuracy, indicating the current 16 $\times$ 24 architecture has insufficient capacity for QAT—larger models (32 $\times$ 48) are being explored. See Table 1.

Integer-Only Implementation and Validation

PTQ-INT8 datapath: INT8 weights/activations, INT32 biases, INT16 adjacency ($K = 2^{12}$). Fixed-point scaling: $y = \text{clamp}((a \cdot s_{\text{fp}} + 2^{M-1}) \gg M)$. *HLS C-simulation validation*: M=24 achieves **0 LSB error** vs Python integer emulator on 8-node subgraphs; M=20 yields ≤ 13 LSB with narrower multipliers. Data-driven bit-width optimization analyzes actual ranges to reduce accumulator/scaling widths (22/21/43 bits vs 32/32/64 conservative defaults), saving 42 bits in critical paths.

Model Variant	Test Accuracy	Parameters	Memory	Status
Base (float)	80.3%	184K	0.70 MB	Complete
Reduced (float)	75.8%	24K	0.09 MB	Complete
PTQ-INT8	75.7%	24K	0.02 MB	HLS C-sim validated
QAT-INT8 (16 $\times$ 24)	28.2%	24K	0.02 MB	In progress

Table 1: Model accuracy on Cora dataset. PTQ preserves accuracy; QAT under development with larger architectures.

Design Space Exploration Framework

The DSE framework is designed to explore: (i) *algorithm level*—hidden dimensions (16-32), fixed-point precision M (20-28 bits); (ii) *quantization level*—unroll factors, accumulator bit-widths; (iii) *HLS level*—pipeline initiation

intervals, resource binding. Each design point executes the full automated flow: PTQ parameter generation $\rightarrow$ test vector creation $\rightarrow$ HLS synthesis $\rightarrow$ report parsing. Pareto analysis will identify non-dominated solutions across accuracy, latency, and resource usage (Fig. 1) [1]. Initial validation demonstrates the methodology on a baseline design point; comprehensive sweeps are in progress.

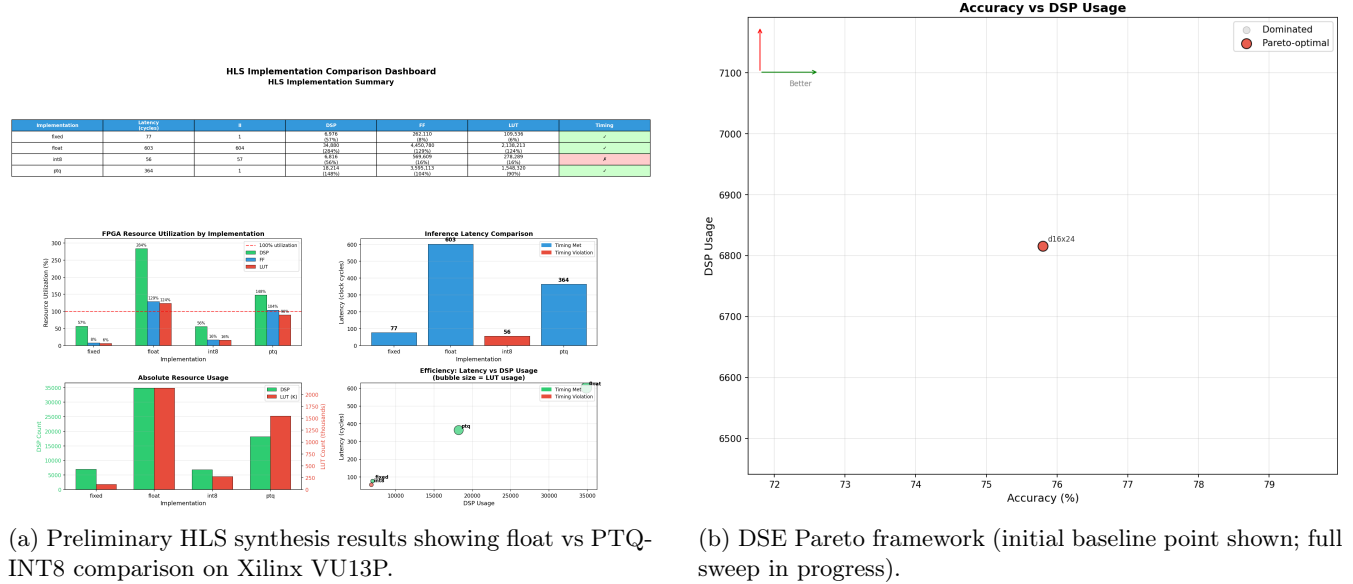


Figure 1: Preliminary results and DSE framework. Full design space exploration is ongoing.

Current Status and Next Steps

Completed: Model training, PTQ-INT8 implementation, HLS C-simulation validation (0 LSB error), automated DSE infrastructure, bit-width optimization tools. *In progress:* QAT with larger architectures, comprehensive DSE parameter sweeps, displaced-muon dataset integration, track-finder graph definition (nodes=DT/CSC/RPC stubs [3], edges= $\Delta\eta$ - $\Delta\phi$ geometric constraints). *Planned:* Retrain on physics data, optimize for 12.5 μ s latency [2], deploy on Phase-2 FPGAs.

Reproducibility

Full workflow automation: `run_pipeline.py` (main pipeline), `explore_design_space.py` (DSE orchestration), `hls/` (HLS kernels). Configuration: `model_config.yaml`, `design_space.yaml`. Verification: `tests/` validate bit-exact Python-HLS agreement at each pipeline stage.

References

References

- [1] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," *arXiv:1706.02216* (2017), <https://arxiv.org/abs/1706.02216>.
- [2] CMS Collaboration, *The Phase-2 Upgrade of the CMS Level-1 Trigger*, CERN-LHCC-2020-004, CMS-TDR-021, Geneva (2020), <https://cds.cern.ch/record/2714892>.
- [3] K. Bunkowski et al., *The Overlap Muon Track Finder of the CMS experiment*, JINST **11**, C03004 (2016), <https://doi.org/10.1088/1748-0221/11/03/C03004>.
- [4] CMS Collaboration, *The CMS trigger system*, JINST **12** (2017) P01020, <https://doi.org/10.1088/1748-0221/12/01/P01020>.